

Porto Taxi Trajectory

Assignment 2 - MySQL

Group: 16

Students: Ingrid Singstad, Olav Larsen Halleraker

1 Introduction

In this assignment, we analysed the Porto Taxi Trajectory dataset, containing approximately 1.7 million taxi trips and 83 million GPS points. The main tasks were to explore and clean the data, design and populate a MySQL database, and answer ten questions about it using SQL and Python.

Group work

At the beginning of the project, we met in person at school to go through the assignment together and agree on a common approach for the exploratory data analysis (EDA), data cleaning, database schema, and queries. We developed the first version of the code together, using AI-assisted development. Because of our different schedules, Ingrid continued with the EDA, data loading, and queries, while Olav completed the remaining parts and finished the project submission.

We stayed in contact throughout the project, and discussed questions, changes, and problems both in person and online. When one of us changed or added something that affected the other parts of the project, we reviewed it to make sure the different components remained consistent. Our main communication channel was Messenger. We also used Git for version control, allowing us to keep track of changes and maintain a shared version of the code. The repository is available at <https://github.com/olavhalleraker/tdt4225-project>.

Setup

We ran MySQL locally in its own Docker container. The programs use Python 3.11 with the packages from the requirements.txt provided, in addition to matplotlib for the EDA figures. DbConnector.py from the assignment material was kept, modified to read host, port, user and password from environment variables. For more information about the setup, see README.md.

File	Purpose
DbConnector.py	Connection to MySQL

cleaning.py	Parsing and cleaning rules shared by the EDA and the insert program
part1_eda.py	EDA of the raw CSV, prints tables and saves figures
part1_insert.py	CREATE TABLE statements and insertion of the cleaned data
part2_queries.py	One method per task with results printed with tabulate
output/	Console output of the three programs, and CSV files with the complete answer lists of tasks 4a, 6, 8 and 9

Table 1: Overview of files in the repository.

2 Results

The results section is divided in three parts following the assignment text given. First, we explain how we performed EDA and our findings from analysing and cleaning the Porto dataset. Then, we describe the schema design and insertion of the cleaned data. Finally, we answer 10 questions using SQL queries and Python. For more details on the implementation, we refer to the Github repository.

2.1 Exploratory data analysis

The EDA is implemented in part1_eda.py. The program reads the raw CSV sequentially with Python's csv module to avoid holding all the rows in memory. For each column it computes the number of empty and non-integer values and the distribution of the values. In addition, every POLYLINE is parsed into GPS points, from which the number of points per trip, the implied speed between consecutive points, and the duration and distance of each trip are derived. Trajectory statistics are computed after removing duplicate TRIP_IDs and applying the same cleaning functions (cleaning.py) as the insertion program, so the reported numbers correspond to the data stored in MySQL. The table below summarises the main findings and how each one is handled during insertion. The complete console output is given in Appendix A, with results from 12 checks referred to as A1-A12.

Attribute	Finding	Handling
TRIP_ID	1 710 670 rows but 1 710 589 unique IDs. 80 IDs occur 2–3 times (81 extra rows). Only 2 groups are exact copies. Usually one copy is a real trajectory and the other has only 0–2 GPS points (75 of the 81 dropped copies) and often another CALL_TYPE / ORIGIN_STAND (Fig. A3).	Keep the copy with the most GPS points (the first one on ties), drop the 81 others.
CALL_TYPE	A 21.3%, B 47.8%, C 30.9%. No unknown codes.	CHAR(1) with a CHECK constraint.

ORIGIN_CALL, ORIGIN_STAND	Empty when not applicable. ORIGIN_CALL is set for type A. 11 302 type-B trips (1.4%) have no ORIGIN_STAND.	Empty string. NULL. B-trips without a stand are kept.
TAXI_ID	448 taxis with 1 to 10 746 trips each (median 3 737). 10 taxis have fewer than 100 trips.	Own table taxi and nothing removed.
TIMESTAMP	Integer Unix time. Converted as UTC, the first trip starts 2013-07-01 00:00:53 and the last 2014-06-30 23:59:56.	Values are converted to DATETIME without any time-zone shift.
DAY_TYPE	A for all 1.7 million trips, also on public holidays such as 25 December (Fig. A5).	Stored unchanged and not used (carries no information).
MISSING_DATA	Only 10 trips are flagged as incomplete.	Kept, flag stored. Their duration is a lower bound.
POLYLINE (length)	5 894 trips have no point, 30 546 one and 7 389 two points, meaning 43 829 invalid trips (2.56%). Median 41 points (10 min), maximum 3 881 (16 h).	All trips stored and generated column is_valid (≥ 3 points). Analyses of trajectories use valid trips.
POLYLINE (GPS errors)	35 points lie outside Portugal/Galicia (e.g. longitude 52.9). 46 985 steps between consecutive points imply more than 200 km/h (many trips start with a stale position and then jump).	Three cleaning rules remove 77 271 points (0.09%) in 38 230 trips.
Duration, distance	Median 10.2 min and 4.0 km. 2 257 trips last more than 2 h (max 970 min, 752 km); 9 074 valid trips move less than 100 m in total.	Kept.
Trip order per taxi	792 of 1.67 million consecutive trip pairs of the same taxi overlap in time (median 26s).	Overlaps count as 0 idle time in task 10.

Missing values

Apart from ORIGIN_CALL and ORIGIN_STAND, which are empty by design, no column has empty values, and all ID and time columns contain valid integers (Fig. A2). The missing data is in the trajectories, e.g. trips without or with very few GPS points,

ten trips flagged with MISSING_DATA, and GPS errors. We keep every non-duplicated trip in the database so that nothing is silently lost and task 7 can be answered from the database, in addition to marking trips with fewer than three points as invalid.

Buggy GPS points

Figure 3 shows the speed implied by consecutive GPS points (15s apart). Most steps are below 100 km/h and the counts fall steeply towards 200 km/h, while above 200 km/h there is a long tail of physically impossible jumps (the last bar collects everything above 400 km/h). Inspecting such trips showed that this was due to single points far off the route, trips that start with one or a few stale positions and then jump to where the taxi really is, and gaps in the recording after which the trajectory continues somewhere else. Therefore, we applied three rules in the clean_points function in cleaning.py:

- **Region rule:** points outside latitude 36.9 to 43.8 and longitude -9.6 to -6.0 (mainland Portugal and Galicia) are dropped.
- **Start rule:** if at most 4 points (one minute) come before an impossible jump and more points follow after it, the starting points are stale positions and are dropped.
- **Speed rule:** a point is kept only if it can be reached from the last kept point without driving faster than 200 km/h (the motorway limit is 120 km/h). A wrong point in the middle is skipped, and after a real gap the trajectory is picked up again as soon as the elapsed time makes the new position reachable.

We compared several variants before choosing these rules. Comparing each point only with the previous kept point (no start rule) fails when the first point is wrong because the correct rest of the trajectory is rejected, and about 270 trips then lose more than half of their points. The opposite, keeping only the longest chain of consistent segments (or dropping any shorter part before the first jump), removed 270 000 to 450 000 points. The final rules remove 77 271 points, and only 123 very noisy trips lose more than half of their points. Removed points are not stored, but point_index keeps the original position, so the time of every point is still $\text{start_time} + 15\text{s} \cdot \text{point_index}$, and the duration is computed from the original number of points.

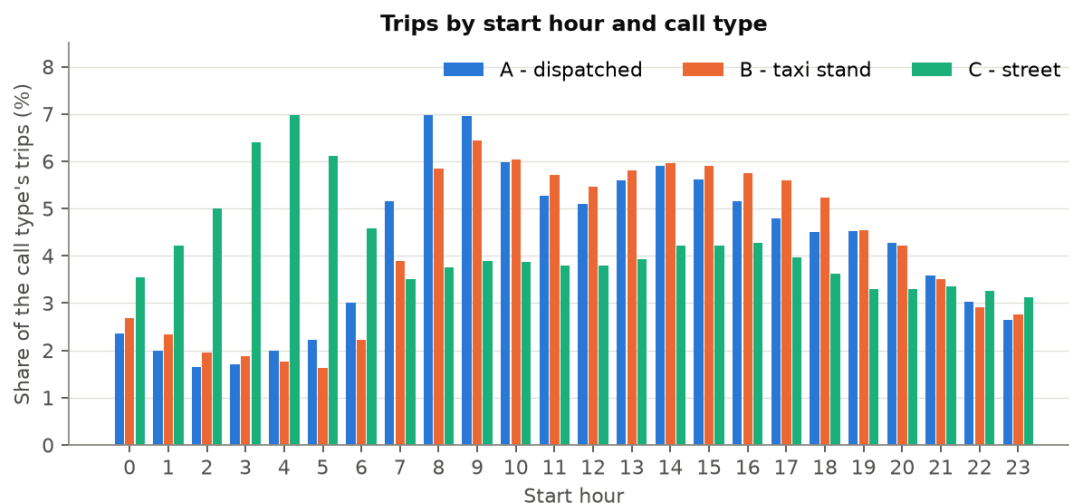


Figure 1: Share of trips per start hour for each call type. Street-hailed trips (C) dominate at night, dispatched (A) and stand trips (B) peak in the morning

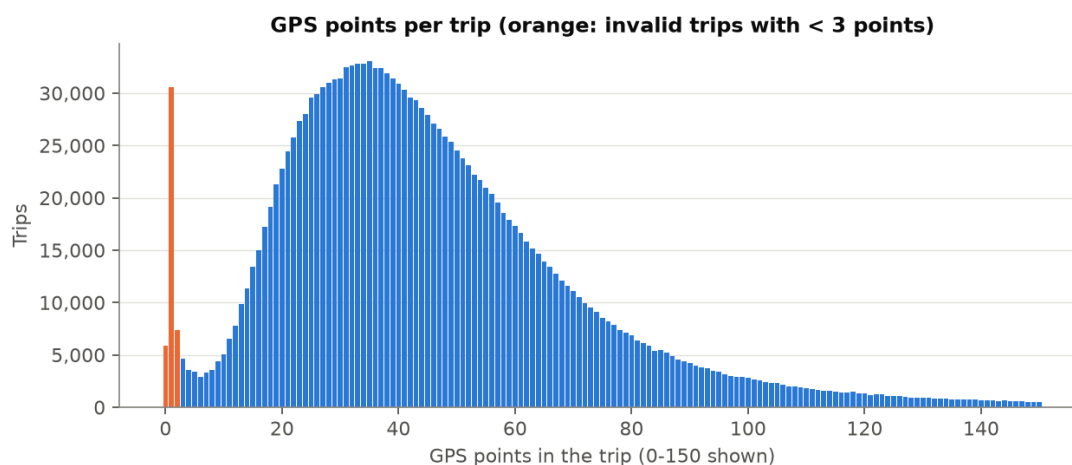


Figure 2: GPS points per trip. The orange bars are the invalid trips with 0, 1 or 2 points.

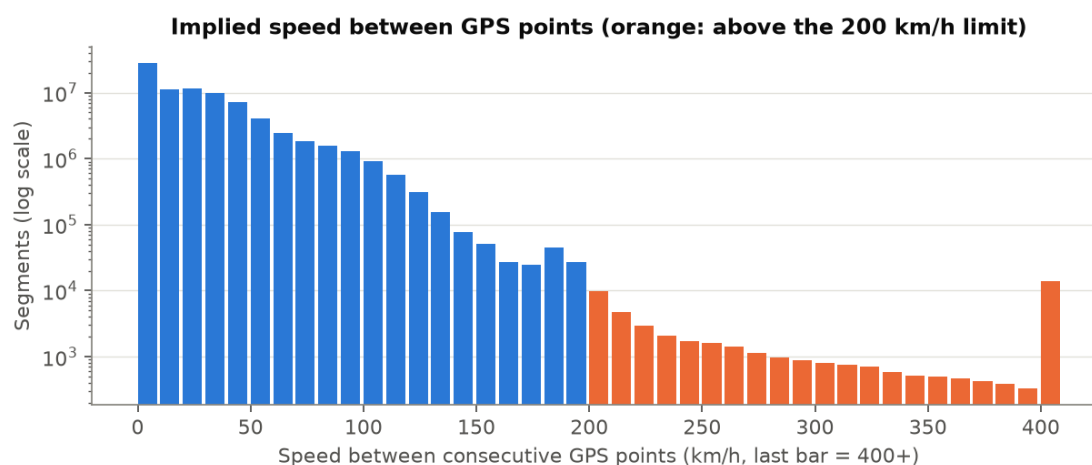


Figure 3: Speed implied by consecutive raw GPS points (log scale). Orange is above the 200 km/h limit.

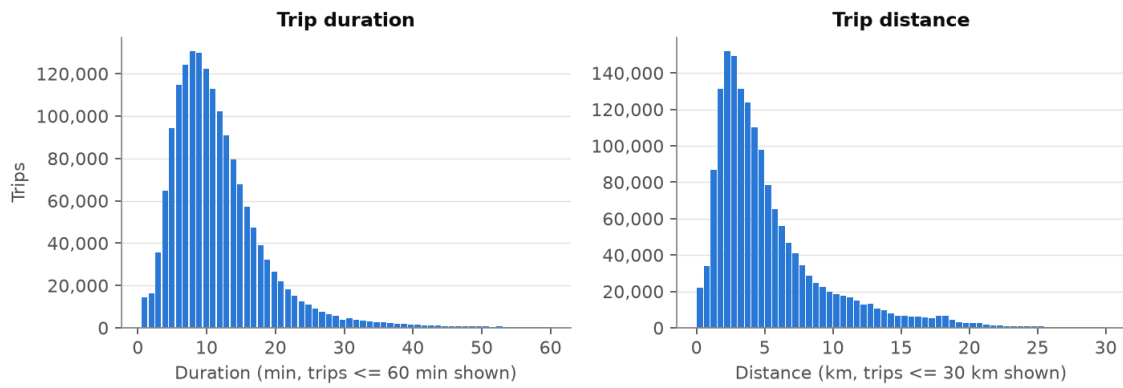


Figure 4: Duration and distance of valid trips after cleaning (the long tails are cut off).

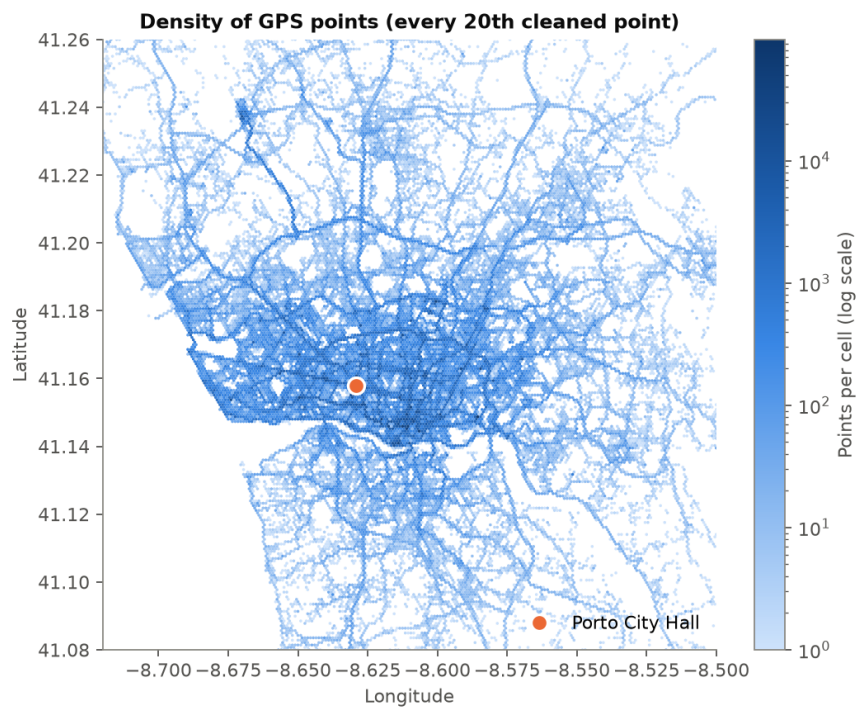


Figure 5: Density of cleaned GPS points in Porto (every 20th point), with Porto City Hall (task 6).

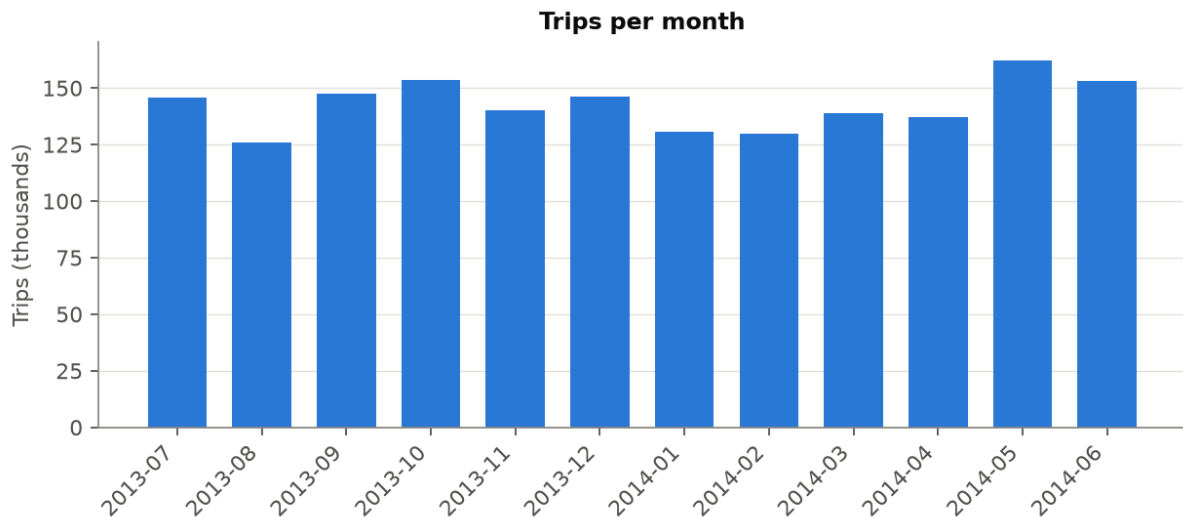


Figure 6: Trips per month.

2.2 Schema and insertion

The data is stored in three tables: a taxi has many trips, and a trip has many GPS points. These CREATE TABLE statements are executed by part1_insert.py shown below.

```
CREATE TABLE taxi (
    taxi_id INT NOT NULL PRIMARY KEY
);

CREATE TABLE trip (
    trip_id          BIGINT          NOT NULL PRIMARY KEY,
    taxi_id          INT             NOT NULL,
    call_type        CHAR(1)         NOT NULL,
    origin_call      INT             NULL,
    origin_stand     SMALLINT        NULL,
    day_type         CHAR(1)         NOT NULL,
    missing_data     BOOLEAN         NOT NULL,
    start_time       DATETIME        NOT NULL,
    end_time         DATETIME        NULL,
    duration_s       INT             NULL,
    n_points_raw     SMALLINT UNSIGNED NOT NULL,
    n_points_clean   SMALLINT UNSIGNED NOT NULL,
    is_valid         BOOLEAN AS (n_points_raw >= 3) STORED,
    distance_km      DOUBLE          NULL,
    start_lat        DECIMAL(9, 6)   NULL,
    start_lon        DECIMAL(9, 6)   NULL,
    end_lat          DECIMAL(9, 6)   NULL,
    end_lon          DECIMAL(9, 6)   NULL,
    CONSTRAINT fk_trip_taxi FOREIGN KEY (taxi_id) REFERENCES taxi (taxi_id)
ON DELETE CASCADE,
    CONSTRAINT chk_call_type CHECK (call_type IN ('A', 'B', 'C')),
    CONSTRAINT chk_day_type CHECK (day_type IN ('A', 'B', 'C')),
    INDEX idx_trip_taxi_start (taxi_id, start_time)
);
```

```
CREATE TABLE gps_point (
    trip_id      BIGINT          NOT NULL,
    point_index  SMALLINT UNSIGNED NOT NULL,
    latitude     DECIMAL(9, 6)   NOT NULL,
    longitude    DECIMAL(9, 6)   NOT NULL,
    PRIMARY KEY (trip_id, point_index),
    CONSTRAINT fk_point_trip FOREIGN KEY (trip_id) REFERENCES trip
(trip_id) ON DELETE CASCADE
);
```

Reasoning behind the design

- One row per GPS point instead of the POLYLINE text. The nested list becomes queryable with plain SQL (e.g. the location search in task 6) without parsing JSON in every query. The primary key (trip_id, point_index) keeps the order of the points, and the time of a point follows from point_index, so we do not store 83 million timestamps.
- Derived trip columns are computed once in Python during the insert. end_time, duration_s, distance_km, the first/last cleaned point and the point counts before and after cleaning. This is redundant with gps_point, but the data never changes after loading, and it keeps the Part 2 queries short and fast (no aggregation over 83 million points per query).
- Invalid trips are kept and marked by the generated column is_valid, so no data is lost.
- Data types: TRIP_ID has 19 digits and needs BIGINT. Coordinates use DECIMAL(9, 6), which stores the six decimals of the source exactly and takes 5 bytes instead of 8 for a DOUBLE. Point counts fit in SMALLINT UNSIGNED (max 3,881).
- Integrity: foreign keys with ON DELETE CASCADE (deleting a taxi or a trip removes its trips/points), CHECK constraints for the code columns, NOT NULL wherever a value must exist. Missing values are NULL, never empty strings or 0.
- Index (taxi_id, start_time) on trip supports the per-taxi questions, especially ordering each taxi's trips in time for task 10.

Insertion

part1_insert.py drops and recreates the tables, inserts the 448 taxis and then streams the CSV again. Each row is cleaned with cleaning.py.build_trip, and trips and their points are inserted in batches of 2 000 trips. The whole insert takes about 12 minutes. Figures 7 and 8 show the log and the check at the end, showing that the row counts match the EDA (1 710 589 trips, 83 332 012 GPS points).


```

Created tables taxi, trip and gps_point
Inserted 448 taxis
100000 trips inserted (43 s)
200000 trips inserted (84 s)
300000 trips inserted (124 s)
400000 trips inserted (165 s)
500000 trips inserted (209 s)
600000 trips inserted (255 s)
700000 trips inserted (297 s)
800000 trips inserted (341 s)
900000 trips inserted (383 s)
1000000 trips inserted (426 s)
1100000 trips inserted (468 s)
1200000 trips inserted (511 s)
1300000 trips inserted (554 s)
1400000 trips inserted (595 s)
1500000 trips inserted (638 s)
1600000 trips inserted (681 s)
1700000 trips inserted (724 s)

Insert finished in 12.1 minutes
Step                                Count
-----
CSV rows                            1,710,670
Duplicate rows skipped              81
Trips inserted                      1,710,589
GPS points in inserted trips        83,409,283
GPS points inserted                 83,332,012
Points outside region (removed)     35
Points removed by start/speed rules 77,236

```

Figure 7: Console output of part1_insert.py.

```

Rows per table:
Table      Rows
-----
taxi       448
trip       1,710,589
gps_point  83,332,012

SELECT trip_id, taxi_id, call_type, origin_call, origin_stand, day_type, missing_data, start_time, end_time FROM trip LIMIT 5:
trip_id      taxi_id      call_type      origin_call      origin_stand      day_type      missing_data      start_time      end_time
-----
1372636853620000380  20000380  C              0              A              0              2013-07-01 00:00:53  2013-07-01 00:06:53
1372636854620000520  20000520  C              0              A              0              2013-07-01 00:00:54  2013-07-01 00:11:24
1372636858620000589  20000589  C              0              A              0              2013-07-01 00:00:58  2013-07-01 00:06:28
1372636875620000233  20000233  C              0              A              0              2013-07-01 00:01:15  2013-07-01 00:06:30
1372636896620000360  20000360  C              0              A              0              2013-07-01 00:01:36  2013-07-01 00:12:06

SELECT trip_id, duration_s, n_points_raw, n_points_clean, is_valid, distance_km, start_lat, start_lon, end_lat, end_lon FROM trip LIMIT 5:
trip_id      duration_s      n_points_raw      n_points_clean      is_valid      distance_km      start_lat      start_lon      end_lat      end_lon
-----
1372636853620000380  360            25              25              1            2.0126495087449223  41.140746      -8.610291      41.145345      -8.605890
1372636854620000520  630            43              39              1            7.248476933422172  41.151951      -8.574678      41.142915      -8.607996
1372636858620000589  330            23              23              1            2.6510520754856137  41.141412      -8.618643      41.154489      -8.630838
1372636875620000233  315            22              22              1            2.2362960148171394  41.148009      -8.619894      41.157351      -8.609490
1372636896620000360  630            43              41              1            5.051409639409595  41.146137      -8.617599      41.169141      -8.636625

SELECT * FROM gps_point LIMIT 5:
trip_id      point_index      latitude      longitude
-----
1372636853620000380  0              41.140746      -8.610291
1372636853620000380  1              41.140755      -8.610300
1372636853620000380  2              41.140890      -8.610309
1372636853620000380  3              41.141358      -8.613657
1372636853620000380  4              41.141484      -8.614602

```

Figure 8: Rows per table and the first rows of each table after the insert.

2.2 Queries

In this part we make the following assumptions: A trip is valid if its POLYLINE has at least 3 points (task 7). Counting questions (tasks 1-3, 4a, 7) use all trips. Questions about duration, distance, place and time (tasks 4b, 5, 6, 8, 9, 10) use valid trips only. Questions with long answers are printed as the count plus the first 20 rows. The complete lists are written to output/.

Task 1: How many taxis, trips and GPS points?

Three COUNT(*) subqueries on the three tables show 448 taxis, 1 710 589 trips (1 666 760 valid) and 83 332 012 GPS points in the database. Before removing GPS errors the trips had 83 409 283 points.

```
=====
Task 1: How many taxis, trips, and total GPS points are there?
=====
  taxis      trips      valid_trips      gps_points_stored      gps_points_before_cleaning
-----
    448  1,710,589    1,666,760          83,332,012          83,409,283
[task 1 took 15.3 s]
```

Task 2: Average number of trips per taxi

The trips are counted per taxi (taxi LEFT JOIN trip, so a taxi without trips would count as 0) and averaged. A taxi has on average 3 818 trips (3 720 valid). The numbers vary a lot, from 1 to 10 731 trips.

```
=====
Task 2: What is the average number of trips per taxi?
=====
  avg_trips_per_taxi      avg_valid_trips_per_taxi      min_trips      max_trips
-----
          3818.28              3720.45              1          10,731
[task 2 took 1.8 s]
```

Task 3: Top 20 taxis with the most trips

Taxi 20000080 has the most trips (10 731), but only 4 211 of them are valid. This taxi has about 6 500 trips with 0-2 GPS points, which suggests a faulty GPS device or a driver who starts and stops the meter without driving. Ranked by valid trips it would not be in the top 20.

```
=====
Task 3: List the top 20 taxis with the most trips.
=====
```

rank	taxi_id	trips	valid_trips
1	20000080	10,731	4,211
2	20000403	9,237	7,516
3	20000066	8,443	5,952
4	20000364	7,821	7,270
5	20000483	7,729	7,623
6	20000129	7,608	7,101
7	20000307	7,497	7,290
8	20000621	7,276	7,250
9	20000089	7,266	6,977
10	20000424	7,176	7,010
11	20000492	7,171	7,115
12	20000529	6,937	6,918
13	20000616	6,924	6,466
14	20000678	6,538	6,475
15	20000372	6,535	6,257
16	20000304	6,505	6,374
17	20000042	6,467	6,437
18	20000325	6,460	6,352
19	20000179	6,430	6,366
20	20000235	6,410	6,387

```
[task 3 took 1.4 s]
```

Task 4a: Most used call type per taxi

For 354 taxis the most used call type is B (taxi stand), for 83 it is C (street) and for 12 it is A (dispatch). Taxi 20000344 has a tie and is counted twice. The full list of 449 rows is in output/task4a_most_used_call_type.csv.

Task 4a: What is the most used call type per taxi?

Number of taxis per most used call type:

most_used_call_type	taxis
---------------------	-------

A	12
B	354
C	83

Taxis with a tie between two call types: 1 [20000344]

First 20 taxis:

taxi_id	most_used_call_type	trips	share_of_taxi_trips_pct
20000001	B	1,311	47.59
20000002	B	1,576	40.73
20000003	C	844	44.47
20000004	B	2,680	54.31
20000005	B	3,034	49.79
20000006	B	1,702	50.61
20000007	B	2,770	49.63
20000008	B	2,665	54.97
20000009	B	2,099	43.96
20000010	B	2,842	47.57
20000011	B	2,622	43.97
20000012	B	2,257	40.70
20000013	B	2,581	51.56
20000014	B	2,523	60.87
20000015	B	2,819	56.80
20000017	B	1,129	41.33
20000018	B	958	54.52
20000020	B	2,304	54.24
20000021	B	1,901	57.38
20000022	B	1,990	49.58

(all 449 rows written to output/task4a_most_used_call_type.csv)

[task 4a took 0.4 s]

Task 4b: Duration, distance and start time bands per call type

Street-hailed trips are the longest and are much more common at night. A third of them start between 00 and 06, compared to 12% for the two other types (see also Figure 1).

Task 4b: For each call type, compute the average trip duration and distance, and also report the share of trips starting in four time bands: 00-06, 06-12, 12-18, and 18-24.

call_type	trips	avg_duration_min	avg_distance_km	pct_00_06	pct_06_12	pct_12_18	pct_18_24
A	362,170	12.60	5.40	11.92	33.38	32.17	22.52
B	807,446	11.23	5.07	12.23	30.13	34.48	23.16
C	497,144	13.67	6.60	33.07	23.25	23.91	19.77

[task 4b took 0.6 s]

Task 5: Taxis with most hours and distance driven

Hours and distance are summed per taxi over valid trips, sorted by hours. Taxi 20000904 is first on both with 1 961 hours and 62 338 km. The other taxis have 1 260 to 1 650 hours. Hours and distance are related but not the same ranking, e.g. taxi 20000621 is 13th by hours but only 63rd by distance, meaning it drives more slowly on average.

```
=====
Task 5: Find the taxis with the most total hours driven as well as total
distance driven. List them in order of total hours.
=====
```

```
Top 20 taxis by total hours (distance_rank = position when sorted by distance):
```

hours_rank	taxi_id	trips	total_hours	total_distance_km	distance_rank
1	20000904	4,925	1961.04	62338.29	1
2	20000129	7,101	1649.56	36331.04	30
3	20000307	7,290	1587.01	39755.60	12
4	20000529	6,918	1507.28	42190.12	5
5	20000276	5,795	1419.81	46375.78	3
6	20000436	5,488	1410.64	44039.58	4
7	20000483	7,623	1397.60	35994.34	33
8	20000372	6,257	1387.10	40365.55	11
9	20000616	6,466	1348.66	35244.98	35
10	20000179	6,366	1324.22	39707.00	13
11	20000574	5,849	1301.84	37167.91	22
12	20000235	6,387	1287.86	36370.55	29
13	20000621	7,250	1282.41	31833.12	63
14	20000364	7,270	1279.83	41463.29	8
15	20000435	5,824	1277.55	40368.30	10
16	20000199	6,136	1270.73	38928.32	14
17	20000446	5,276	1266.62	34736.63	38
18	20000011	5,919	1266.09	38339.63	19
19	20000395	5,481	1263.78	34047.13	44
20	20000492	7,115	1262.53	32975.09	53

```
[task 5 took 0.4 s]
```

Task 6: Trips within 100m of Porto City Hall

MySQL first selects the points in a small bounding box around the City Hall and Python computes the exact haversine distance for these candidates and keeps the closest point per trip. 637 211 points lie in the box, 309 679 of them within 100m. 126 173 trips passed within 100m of the City Hall (7.6% of the valid trips). All are listed in output/task6_trips_near_city_hall.csv.

```
=====
Task 6: Find the trips that passed within 100 m of Porto City Hall (longitude,
latitude) = (-8.62911, 41.15794).
=====
```

```
GPS points in the bounding box: 637,211, of them within 100 m: 309,679
```

```
Trips that passed within 100 m of Porto City Hall: 126,173
```

```
First 20 (by start time):
```

trip_id	taxi_id	start_time	closest_distance_m
1372636896620000360	20000360	2013-07-01 00:01:36	95.60
1372636956620000167	20000167	2013-07-01 00:02:36	84.90
1372637423620000341	20000341	2013-07-01 00:10:23	96.40
1372637610620000497	20000497	2013-07-01 00:13:30	91.80
1372637721620000678	20000678	2013-07-01 00:15:21	73.90
1372638361620000154	20000154	2013-07-01 00:26:01	98.20
1372638851620000406	20000406	2013-07-01 00:34:11	95.90
1372639364620000015	20000015	2013-07-01 00:42:44	67.70
1372639441620000012	20000012	2013-07-01 00:44:01	80.20
1372640338620000423	20000423	2013-07-01 00:58:58	93.00
1372641197620000653	20000653	2013-07-01 01:13:17	82.90
1372641991620000231	20000231	2013-07-01 01:26:31	80.40
1372642071620000305	20000305	2013-07-01 01:27:51	97.10
1372642706620000653	20000653	2013-07-01 01:38:26	90.60
1372643415620000653	20000653	2013-07-01 01:50:15	83.40
1372643825620000569	20000569	2013-07-01 01:57:05	97.80
1372646982620000600	20000600	2013-07-01 02:49:42	86.00
1372648688620000308	20000308	2013-07-01 03:18:08	98.90
1372649313620000231	20000231	2013-07-01 03:28:33	97.90
1372650736620000012	20000012	2013-07-01 03:52:16	86.10

```
(all 126,173 rows written to output/task6_trips_near_city_hall.csv)
```

```
[task 6 took 42.6 s]
```

Task 7: Invalid trips

There are 43 829 invalid trips (2.56%). 5 894 without any GPS point, 30 546 with one and 7 389 with two. In addition, 400 valid trips have fewer than three points left after the GPS errors are removed.

```
=====
Task 7: Identify the number of invalid trips (fewer than 3 GPS points).
=====
```

gps_points	trips
0	5,894
1	30,546
2	7,389
total	43,829

```
Invalid trips: 43,829 of 1,710,589 (2.56 %)
```

```
In addition, 400 valid trips have fewer than 3 points left after removing GPS errors.
```

```
[task 7 took 0.5 s]
```

Task 8: Midnight crossers

A valid trip crosses midnight when its end date is the day after its start date. 8 074 trips (0.48% of the valid trips) started on one day and ended on the next. No trip lasts long enough to span two midnights.

```
=====
Task 8: Find the trips that started on one calendar day and ended on the next
(midnight crossers).
=====
Midnight-crossing trips: 8,074

First 20:
trip_id      taxi_id  call_type  start_time      end_time      duration_min
-----
1372722134620000435  20000435  C          2013-07-01 23:42:14  2013-07-02 00:02:44      20.50
1372722457620000569  20000569  C          2013-07-01 23:47:37  2013-07-02 00:28:22      40.75
1372722471620000540  20000540  A          2013-07-01 23:47:51  2013-07-02 00:02:21      14.50
1372722555620000680  20000680  C          2013-07-01 23:49:15  2013-07-02 00:12:30      23.25
1372722610620000297  20000297  B          2013-07-01 23:50:10  2013-07-02 00:04:55      14.75
1372722646620000351  20000351  B          2013-07-01 23:50:46  2013-07-02 00:00:31       9.75
1372722663620000108  20000108  C          2013-07-01 23:51:03  2013-07-02 00:03:48      12.75
1372722796620000451  20000451  A          2013-07-01 23:53:16  2013-07-02 00:02:01       8.75
1372722801620000015  20000015  B          2013-07-01 23:53:21  2013-07-02 00:05:36      12.25
1372722877620000598  20000598  B          2013-07-01 23:54:37  2013-07-02 00:04:07       9.50
1372722880620000005  20000005  B          2013-07-01 23:54:40  2013-07-02 00:09:10      14.50
1372722892620000307  20000307  C          2013-07-01 23:54:52  2013-07-02 00:08:22      13.50
1372722919620000603  20000603  A          2013-07-01 23:55:19  2013-07-02 00:01:49       6.50
1372722923620000273  20000273  B          2013-07-01 23:55:23  2013-07-02 00:01:53       6.50
1372722970620000195  20000195  C          2013-07-01 23:56:10  2013-07-02 00:00:10       4.00
1372722985620000454  20000454  A          2013-07-01 23:56:25  2013-07-02 00:05:25       9.00
1372723001620000483  20000483  C          2013-07-01 23:56:41  2013-07-02 00:02:56       6.25
1372723054620000014  20000014  A          2013-07-01 23:57:34  2013-07-02 00:09:04      11.50
1372723060620000337  20000337  C          2013-07-01 23:57:40  2013-07-02 00:03:55       6.25
1372723069620000688  20000688  A          2013-07-01 23:57:49  2013-07-02 00:10:49      13.00
(all 8,074 rows written to output/task8_midnight_crossers.csv)
[task 8 took 0.4 s]
```

Task 9: Circular trips

MySQL returns the first and last cleaned point of every valid trip with at least three cleaned points and Python checks $\text{haversine}(\text{start}, \text{end}) \leq 50\text{m}$. 21 514 trips (1.29%) start and end within 50m of each other. Nuancing the results, we observe that 7 798 of them drove less than 100m in total (the taxi did not move, average 2.5 min), while 11 993 drove at least 1 km and are real round trips (average 31.5 min).

```
=====
Task 9: Find the trips whose start and end points are within 50 m of each
other (circular trips).
=====
```

```
Circular trips (start and end within 50 m): 21,514 of 1,666,360 trips checked (1.29 %)
```

```

circular trips that ...      trips      avg_duration_min
-----
drove < 100 m (taxi did not move)    7,798            2.48
drove 100 m - 1 km                   1,723           14.02
drove >= 1 km (real round trip)     11,993          31.48

```

```
First 20 (by start time):
```

```

trip_id      taxi_id      start_time      duration_min      distance_km      start_end_distance_m
-----
1372638303620000112    20000112    2013-07-01 00:25:03        28.75        8.63        2.50
1372639092620000233    20000233    2013-07-01 00:38:12         1.75        0.02        8.80
1372644642620000305    20000305    2013-07-01 02:10:42        37.50        1.21       40.90
1372650711620000403    20000403    2013-07-01 03:51:51         2.75        0.02        6.70
1372650908620000403    20000403    2013-07-01 03:55:08         0.50        0.00        2.30
1372661911620000496    20000496    2013-07-01 06:58:31         3.75        0.54       11.00
1372662261620000031    20000031    2013-07-01 07:04:21        21.75        9.53        2.50
1372663272620000206    20000206    2013-07-01 07:21:12        91.00        8.00       13.00
1372664679620000186    20000186    2013-07-01 07:44:39         6.50        0.09       13.80
1372664787620000397    20000397    2013-07-01 07:46:27        30.00        6.24       17.80
1372665027620000574    20000574    2013-07-01 07:50:27        40.75        8.47       23.00
1372667532620000443    20000443    2013-07-01 08:32:12         3.50        0.03        4.30
1372667871620000152    20000152    2013-07-01 08:37:51         8.00        1.02       18.50
1372667893620000116    20000116    2013-07-01 08:38:13         8.25        0.16       27.30
1372668421620000093    20000093    2013-07-01 08:47:01        18.00       15.20       34.90
1372669341620000388    20000388    2013-07-01 09:02:21        34.25        4.89        0.80
1372669439620000460    20000460    2013-07-01 09:03:59        20.00        5.00       21.10
1372669656620000184    20000184    2013-07-01 09:07:36         7.75        2.76       13.80
1372669944620000398    20000398    2013-07-01 09:12:24        26.25        7.27        5.50
1372669996620000616    20000616    2013-07-01 09:13:16        33.00        8.01        5.40

```

```
(all 21,514 rows written to output/task9_circular_trips.csv)
```

```
[task 9 took 5.4 s]
```

Task 10: Average idle time between consecutive trips

Taxi 20000941 has the highest average idle time with 1 747 hours (73 days), followed by 20000969 with 282 hours. Both have very few trips (4 and 25), so a single long break dominates their average. The regular taxis in the list average 5-13 hours between trips, because the average includes nights and days off (longest_idle_days). The last column shows the idle time when only gaps shorter than 6 hours (breaks within a shift) are counted, showing about 45-100 minutes for the regular taxis.


```
=====
Task 10: For each taxi, compute the average idle time between consecutive
trips. List the top 20 taxis with the highest average idle time.
=====
rank  taxi_id  idle_periods  avg_idle_hours  longest_idle_days  avg_idle_min_gaps_under_6h
-----
1  20000941      3      1746.61      218.32      1.96
2  20000969     24      282.07      179.99      37.70
3  20000312    581      13.12      33.91      55.13
4  20000510    634      12.96      36.72      61.75
5  20000609    557      11.94      17.72      56.37
6  20000579    743      11.43      34.54      56.78
7  20000079     97       9.70       9.95      67.92
8  20000072    915       9.34       5.95      98.21
9  20000185    668       9.01      84.98      55.41
10 20000449    962       8.85      15.66      66.68
11 20000902   1,046       7.51      10.62      86.20
12 20000170      6       6.73       0.97      66.41
13 20000535   1,275       6.64      23.66      90.52
14 20000315   1,308       6.45      11.70      97.41
15 20000225   1,348       6.28     189.29      66.18
16 20000071   1,354       6.25       4.62      83.40
17 20000545   1,243       5.92      53.67      70.25
18 20000407   1,440       5.64      18.61      60.12
19 20000205   1,319       5.57       5.67      46.70
20 20000443   1,551       5.42       4.79      62.40
[task 10 took 5.4 s]
```

3 Discussion

3.1 Choices that go beyond the assignment text

Time zone

The assignment says to convert `TIMESTAMP` to `DATETIME`, but not in which time zone. We have read it as UTC since the dataset starts and ends at midnight and concluded that the values already encode Porto time. Converting with the Europe/Lisbon time zone would move every summer trip one hour later and change the time bands (task 4b) and the midnight crossers (task 8).

Which trips to use

Counting questions use all trips and questions about duration, distance, place and time use valid trips only (see Part 2). Task 4a counts all trips, since the call type is known even without GPS points.

3.2 Pain points and what we learned

Because of the size of the data, we had to stream the CSV instead of loading everything into memory. The GPS points were inserted in batches with one commit per batch, which took about 12 minutes in total. Duplicate `TRIP_ID`s also had to be resolved before inserting because they would violate the primary key. Finding reasonable GPS cleaning rules took several attempts, and we learned that a column can be complete without being useful, e.g. `DAY_TYPE` has no missing values but contains only A.

Overall, most of the work was understanding and cleaning the data rather than writing the SQL queries. This surprised both of us. Once the schema contained useful derived values such as duration, distance, validity, and start/end points, most questions became relatively short queries.

We also saw how much assumptions matter. Choices such as the time zone or whether invalid trips are included can change the results. In a real project, there would be even more to consider: documentation may be incomplete or wrong, errors may not be clearly flagged, and different reasonable assumptions can lead to different answers. This made us realize that assumptions should be checked against the data and documented clearly.

4 Feedback

- The column description says DAY_TYPE marks holidays, but it is A for every trip in the data. This is a nice EDA finding but could be mentioned if not intended.
- Overall the assignment was well made. It was especially interesting to work with a real, messy dataset of this size. The duplicate IDs, missing trajectories and GPS errors made the EDA and cleaning feel like real work, and the 83 million GPS points forced us to think about how the data is stored and loaded, not only about the queries.
- A small detail is that the deadline is given as Oct 9 on page 1 and Oct 10 on page 9 of the assignment text.

5 AI use

We used Claude Code, an AI coding assistant during the assignment, with the assignment text and provided files as context. We used it together to plan the EDA checks, the cleaning rules, the schema and the queries, Parts of our implementation is built with its suggestions as inspiration. In addition, Claude Code was used to improve code quality. For the report, Claude gave feedback on language, formulations and mistakes.

Appendix A: EDA console output

Complete output of part1_eda.py. Output from check 1 is referred to as Fig A1 etc.

1. Overview

```
File      data/porto.csv
Size      1.94 GB
Rows (trips) 1,710,670
Columns   TRIP_ID, CALL_TYPE, ORIGIN_CALL, ORIGIN_STAND, TAXI_ID, TIMESTAMP, DAY_TYPE, MISSING_DATA, POLYLINE
First trip start 2013-07-01 00:00:53
Last trip start 2014-06-30 23:59:56
```

2. Missing / empty values per column

Column	Empty values	Share	Non-integer values
TRIP_ID	0	0.00 %	0
CALL_TYPE	0	0.00 %	-
ORIGIN_CALL	1,345,900	78.68 %	0
ORIGIN_STAND	904,091	52.85 %	0
TAXI_ID	0	0.00 %	0
TIMESTAMP	0	0.00 %	0
DAY_TYPE	0	0.00 %	-
MISSING_DATA	0	0.00 %	-
POLYLINE	5,901	0.34 %	-

(POLYLINE counts '[]', i.e. a trip without any GPS point, as empty.)

3. TRIP_ID uniqueness

```
Unique TRIP_IDs      1,710,589
TRIP_IDs that occur more than once 80
Extra (duplicate) rows 81
Duplicate groups that are exact copies 2
Groups where POLYLINE differs 77
Groups where CALL_TYPE differs 23
Groups where ORIGIN_STAND differs 18
Groups where ORIGIN_CALL differs 5
Groups where TAXI_ID / TIMESTAMP differ 0
```

GPS points in the dropped copies: 0 points: 7, 1 points: 63, 2 points: 5, 3+ points: 6

Examples (kept copy = the one with most GPS points):

TRIP_ID	CALL_TYPE	ORIGIN_CALL	ORIGIN_STAND	GPS points	Decision
1372702836620000080	B	NULL	16	2	dropped
1372702836620000080	C	NULL	NULL	54	kept
1373025987620000601	A	2002	NULL	0	dropped
1373025987620000601	C	NULL	NULL	37	kept
1373210896620000598	B	NULL	57	1	dropped
1373210896620000598	C	NULL	NULL	19	kept
1373551710620000014	B	NULL	49	2	dropped
1373551710620000014	C	NULL	NULL	67	kept

4. CALL_TYPE and its relation to ORIGIN_CALL / ORIGIN_STAND

CALL_TYPE	Trips	Share
A	364,770	21.3 %
B	817,881	47.8 %
C	528,019	30.9 %

CALL_TYPE	ORIGIN_CALL set	ORIGIN_STAND set	Trips
A	yes	no	364,770
B	no	no	11,302
B	no	yes	806,579
C	no	no	528,019

-> ORIGIN_CALL is set exactly for type A. ORIGIN_STAND is never set for A/C, but 11,302 type-B trips (1.4 % of B) have no ORIGIN_STAND (kept as NULL).

5. DAY_TYPE

DAY_TYPE	Trips
A	1,710,670

DAY_TYPE on Portuguese public holidays (should have been 'B'):

Date	Holiday	DAY_TYPE counts
2013-08-15	Assumption Day	{'A': 3753}
2013-12-08	Immaculate Conception	{'A': 4904}
2013-12-25	Christmas Day	{'A': 2185}
2014-01-01	New Year's Day	{'A': 7493}
2014-04-18	Good Friday	{'A': 4884}
2014-04-25	Freedom Day	{'A': 4893}
2014-05-01	Labour Day	{'A': 4682}
2014-06-10	Portugal Day	{'A': 3757}
2014-06-24	St. John (Porto holiday)	{'A': 6390}

-> DAY_TYPE is 'A' for every trip, also on holidays, so the column carries no information.

6. MISSING_DATA

```
MISSING_DATA    Trips
-----
False           1,710,660
True            10
```

Trips flagged with missing GPS points:

TRIP_ID	CALL_TYPE	TAXI_ID	GPS points
1374554455620000625	B	20000625	43
1375863510620000454	B	20000454	1
1378544246620000057	B	20000057	11
1381233613620000387	C	20000387	483
1386346894620000904	C	20000904	285
1387137779620000640	C	20000640	51
1388351478620000678	A	20000678	27
1390005983620000640	C	20000640	14
1396631707620000163	C	20000163	1
1399405185620000508	C	20000508	53

7. TAXI_ID

```
-----
Distinct taxis      448
Trips per taxi: min    1
Trips per taxi: median 3737
Trips per taxi: mean   3818.5
Trips per taxi: max    10746
Taxis with < 100 trips 10
-----
```

8. TIMESTAMP (start of trip)

Month	Trips
-----	-----
2013-07	145,640
2013-08	125,861
2013-09	147,514
2013-10	153,394
2013-11	140,024
2013-12	146,258
2014-01	130,875
2014-02	129,872
2014-03	138,969
2014-04	137,061
2014-05	162,107
2014-06	153,095

Weekday	Trips
-----	-----
Mon	229,948
Tue	234,734
Wed	232,617
Thu	247,025
Fri	277,313
Sat	258,636
Sun	230,397

9. POLYLINE - GPS points per trip (after removing duplicate rows)

GPS points	Trips	Share
-----	-----	-----
0 (empty)	5,894	0.34 %
1	30,546	1.79 %
2	7,389	0.43 %
3-9	25,728	1.50 %
10-49	1,012,155	59.17 %
50-99	535,077	31.28 %
100-499	91,729	5.36 %
500-999	1,713	0.10 %
>= 1000	358	0.02 %

-----	-----
Trips	1,710,589
GPS points in total	83,409,283
Points per trip: median / mean / max	41 / 48.8 / 3881
Invalid trips (< 3 GPS points)	43,829 (2.56 %)
-----	-----

10. GPS errors (cleaning rules, see cleaning.py)

```

Points outside the region (lat 36.9..43.8, lon -9.6..-6.0) 35
Points removed by the start/speed rules (> 200 km/h jump) 77,236
Points removed in total 77,271 (0.093 %)
Points kept 83,332,012
Trips with at least one removed point 38,230
Trips that lost more than half of their points 123
Valid trips with < 3 points left after cleaning 400

```

```

Speed between consecutive raw GPS points (15 s apart):
Speed      Segments      Share
-----
0-49 km/h  68,163,127  83.426 %
50-99 km/h  11,282,737  13.809 %
100-149 km/h  2,035,663  2.491 %
150-199 km/h  176,076  0.216 %
>= 200 km/h  46,985  0.058 %

```

11. Duration and distance of valid trips (>= 3 GPS points)

```

      Duration (min)      Distance (km)
-----
p1          1.5          0.33
p25         7          2.47
p50        10.2         4.01
p75        14.5         6.62
p99        47.5        23.96
max        970       751.59
mean       12.3         5.6

```

```

Trips longer than 2 hours  2,257
Trips longer than 6 hours  135
Trips that moved less than 100 m in total  9,074

```

12. Overlapping trips of the same taxi

```

Consecutive trip pairs (same taxi) 1,666,318
Pairs where the next trip starts before the previous ends 792
Median overlap (s) 26.0
90th percentile overlap (s) 93.0

```